

Lab 3: Schema Design — High-Volume Patterns & Hotspot Avoidance (70 min)

Pairs with the [Schema Patterns Playbook](#). Each part of this lab corresponds to one named pattern in the Playbook — that's your take-home reference.

Learning Objectives

By the end of this lab you will be able to:

- Observe a write hotspot live, including its signature in `SHOW RANGES` and on the Hot Ranges dashboard
- Compare insert throughput across four PK strategies: `SERIAL`, `UUID`, composite (`account_id`, `created`), and hash-sharded composite
- Use `SHOW RANGES` and `crdb_internal.ranges_no_leases` to diagnose distribution issues
- Apply pre-splits with `ALTER TABLE ... SPLIT AT` to seed range distribution before a bulk load
- Build an **Append-Only Event Log** with TTL for automatic expiry
- Build the **Outbox Pattern** for atomic event publishing with CDC
- Read the cluster's "leaseholder placement" decisions and explain them
- Build a quick mental shortcut for spotting a likely-problematic PK in code review

Prerequisites

- **Docker Desktop** (or Docker Engine) running — there is no `cockroach` binary to install
- Familiarity with `EXPLAIN` and basic SQL (Lab 4 will deep-dive `EXPLAIN`)

Setup

Start a fresh 3-node demo cluster so range counts are clean from the start:

```
scripts/crdb up      # start the 3-node cluster (skip if it is already running)
scripts/crdb sql     # open a SQL shell
```

Everything runs in Docker — see [Lab 1](#) for the cluster layout. Use `scripts/crdb.sh` on macOS, Linux, and **WSL** — WSL is Linux, so the shell script is the right one there. Only a native Windows shell (`cmd` or PowerShell) needs `scripts\crdb.bat`.

Note the Web UI URL. We'll watch the **Hot Ranges** page (under Advanced Debug → Hot Ranges) live during inserts.

```
CREATE DATABASE hotspots;
USE hotspots;
SET sql_safe_updates = off;           -- lets us TRUNCATE without WHERE
```

Tasks

Part A: Build a Hotspot With SERIAL (Anti-Pattern) (15 min)

1. Create the classic-anti-pattern table:

```
CREATE TABLE events_serial (
  id          SERIAL PRIMARY KEY,
  payload     STRING NOT NULL,
  created     TIMESTAMPTZ DEFAULT now()
);
```

2. Insert 50,000 rows in 5 batches — enough to push the table past the default 512 MiB range size for the right column widths, and certainly enough to demonstrate hotspotting in the QPS:

```
-- Repeat 5 times
INSERT INTO events_serial (payload)
SELECT repeat('x', 400)
FROM generate_series(1, 10000);
```

3. Inspect the resulting ranges:

```
SHOW RANGES FROM TABLE events_serial WITH DETAILS;
SELECT count(*) AS range_count, sum(range_size_mb) AS size_mb
FROM [SHOW RANGES FROM TABLE events_serial WITH DETAILS];
```

On a small enough payload you may see only 1–2 ranges. Crucially, *all recent writes hit the rightmost range*.

4. Look at the IDs:

```
SELECT min(id), max(id), max(id) - min(id) AS spread
FROM events_serial;
```

SERIAL in CockroachDB defaults to `unique_rowid()` — IDs are large 64-bit integers that mix time + node ID. They're *roughly monotonic per node*, so writes still cluster on the recent range.

5. In the Web UI, open Hot Ranges. Filter to events_serial. One range dominates QPS.

Part B: UUID — Random Distribution (*Playbook baseline*) (10 min)

1. Parallel table, recommended pattern:

```
CREATE TABLE events_uuid (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  payload     STRING NOT NULL,  
  created     TIMESTAMPTZ DEFAULT now()  
);  
  
-- Repeat 5 times  
INSERT INTO events_uuid (payload)  
SELECT repeat('x', 400)  
FROM generate_series(1, 10000);
```

2. Compare the range distribution:

```
SELECT count(*) AS range_count FROM [SHOW RANGES FROM TABLE events_uuid];  
SHOW RANGES FROM TABLE events_uuid WITH DETAILS;
```

Multiple ranges, each holding a slice of the random UUID keyspace. The Hot Ranges page shows traffic spread across them.

3. Time the inserts head to head:

```
\set show_times  
  
INSERT INTO events_serial (payload)  
SELECT repeat('x', 400) FROM generate_series(1, 10000);  
  
INSERT INTO events_uuid (payload)  
SELECT repeat('x', 400) FROM generate_series(1, 10000);
```

On a healthy laptop demo cluster the UUID version is faster, and the gap widens as you add concurrency or nodes.

Part C: Composite Primary Key (*Playbook #3 — Per-Tenant Co-located PK*) (10 min)

Multi-tenant SaaS apps usually want all of one tenant's data adjacent in the keyspace.

1. Create the table:

```
CREATE TABLE events_composite (  
  account_id  UUID,  
  event_id    UUID DEFAULT gen_random_uuid(),  
  payload     STRING NOT NULL,  
  created     TIMESTAMPTZ DEFAULT now(),  
  PRIMARY KEY (account_id, event_id)  
);
```

2. Insert 10,000 events each for 50 accounts:

```

WITH accounts AS (
  SELECT gen_random_uuid() AS id FROM generate_series(1, 50)
)
INSERT INTO events_composite (account_id, payload)
SELECT (SELECT id FROM accounts ORDER BY random() LIMIT 1),
       repeat('x', 400)
FROM generate_series(1, 500000); -- ~10,000 events per account on average

```

(This is the largest insert in the lab — give it ~20–30 seconds.)

3. Confirm rows are clustered by account in the keyspace:

```
SHOW RANGES FROM TABLE events_composite WITH DETAILS;
```

Ranges split on `(account_id, ...)` boundaries. One tenant's events land in (typically) one range.
Queries filtering by `account_id` become single-range scans.

4. Confirm with an EXPLAIN:

```

EXPLAIN
SELECT * FROM events_composite
WHERE account_id = (SELECT account_id FROM events_composite LIMIT 1);

```

You should see a `scan` with a span limited to one account's key prefix.

5. What if one tenant is *very* hot?

```

-- All writes from one big tenant
WITH big AS (
  SELECT account_id FROM events_composite
  GROUP BY account_id
  ORDER BY count(*) DESC LIMIT 1
)
INSERT INTO events_composite (account_id, payload)
SELECT (SELECT account_id FROM big), repeat('x', 400)
FROM generate_series(1, 50000);

```

Now Hot Ranges shows that single tenant's range under heavy load. Composite PK alone isn't a hotspot cure — only an even distribution of writes across accounts gives you even load.

Part D: Hash-Sharded Composite (Playbook #1 — Hash-Sharded Time-Series PK) (15 min)

For genuinely time-ordered access (recent events, time-range queries), use hash sharding.

1. Create with `USING HASH`:

```
CREATE TABLE events_sharded (
  account_id UUID,
  created    TIMESTAMPTZ DEFAULT now(),
  id         UUID NOT NULL DEFAULT gen_random_uuid(),
  payload    STRING NOT NULL,
  PRIMARY KEY (account_id, created, id) USING HASH WITH (bucket_count = 16)
);
```

⚠ **id is not decoration.** `now()` is the **transaction** timestamp — every row inserted by a single statement gets the *same* value. Without a per-row tiebreaker, `(account_id, created)` is not unique and the very next step fails with `duplicate key value violates unique constraint "events_sharded_pkey"`. Use a UUID (as here) or `clock_timestamp()` when you need genuine per-row wall-clock time.

2. **Insert 20,000 events for ONE account** — what was a hotspot in Part C is now spread:

```
WITH hot_account AS (
  SELECT '00000000-0000-0000-0000-000000000001'::UUID AS id
)
INSERT INTO events_sharded (account_id, payload)
SELECT id, repeat('x', 400)
FROM hot_account, generate_series(1, 20000);
```

3. **Look at ranges:**

```
SHOW RANGES FROM TABLE events_sharded WITH DETAILS;
```

You should see multiple ranges, each holding a slice of one of the 16 hash buckets.

4. **Verify range queries still work efficiently:**

```
EXPLAIN ANALYZE
SELECT *
FROM events_sharded
WHERE account_id = '00000000-0000-0000-0000-000000000001'::UUID
      AND created > now() - INTERVAL '5 minutes'
ORDER BY created DESC
LIMIT 10;
```

The plan queries all 16 buckets in parallel and merges results. Each bucket scan is small; total cost stays bounded.

5. **Compare bucket counts:** Drop and recreate with `bucket_count = 4` and 64; rerun the same inserts. How does the range count change? When is too-many-buckets bad?

Rule of thumb: pick `bucket_count` to match your concurrency, not your data size. 16 is usually right; > 64 is rarely useful.

Part E: Pre-Splits (*Playbook #10 — Pre-Split + Bulk Import*) (10 min)

Sometimes you know up front your keys will be sequential (e.g., a one-time import). Pre-splitting tells CockroachDB to create ranges in advance.

1. Create a sequential-key table:

```
CREATE TABLE big_import (  
  id      INT8 PRIMARY KEY,  
  payload STRING NOT NULL  
);
```

2. Pre-split it at 10 evenly-spaced boundaries:

```
ALTER TABLE big_import SPLIT AT  
VALUES (100000), (200000), (300000), (400000), (500000),  
        (600000), (700000), (800000), (900000);
```

3. Confirm the ranges exist:

```
SHOW RANGES FROM TABLE big_import WITH DETAILS;
```

You should see 10 ranges, each empty.

4. Bulk-insert across the keyspace (round-robin so we hit every range):

```
INSERT INTO big_import (id, payload)  
SELECT g, repeat('x', 200)  
FROM generate_series(1, 1000000) g;
```

On the Hot Ranges page, you should see load spread across all 10 pre-split ranges. Compare to the same insert against `events_serial` from Part A — much more even.

5. Inspect the leaseholder placement:

```
SELECT lease_holder, count(*) AS ranges  
FROM [SHOW RANGES FROM TABLE big_import WITH DETAILS]  
GROUP BY lease_holder;
```

Roughly even? If not, the allocator may not yet have rebalanced; wait a minute and re-check.

Part G: Append-Only Event Log with TTL (*Playbook #2, #9*) (10 min)

For high-volume audit logs, metric streams, and anything that's "write once, read recently, age out":

1. Create the event log with a TTL clause:

```
CREATE TABLE event_log (
  id        UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  ts        TIMESTAMPTZ DEFAULT now(),
  payload   JSONB
) WITH (ttl_expire_after = '30 days', ttl_job_cron = '@hourly');
```

Random UUID PK distributes writes; the TTL job sweeps anything older than 30 days every hour.

2. Insert 50,000 fake events:

```
INSERT INTO event_log (payload)
SELECT jsonb_build_object('user', g, 'action', 'view')
FROM generate_series(1, 50000) g;
```

3. Inspect the TTL job:

```
SELECT job_id, status, description
FROM [SHOW JOBS]
WHERE description ILIKE '%ttl%event_log%'
ORDER BY created DESC LIMIT 5;
```

You'll see a recurring `ROW LEVEL TTL` job for `event_log`. It runs on your `@hourly` cron.

4. Set a tiny TTL on a test table to see the sweep work in real time:

```
CREATE TABLE ttl_demo (
  id        UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  ts        TIMESTAMPTZ DEFAULT now()
) WITH (ttl_expire_after = '30 seconds', ttl_job_cron = '* * * * *');

-- list the column: SELECT NULL, now() would try to write NULL into the PK
INSERT INTO ttl_demo (ts) SELECT now() FROM generate_series(1, 100);
SELECT count(*) FROM ttl_demo;           -- 100

-- Wait ~90 seconds for TTL to expire + a job run
SELECT pg_sleep(90);
SELECT count(*) FROM ttl_demo;           -- 0 (rows swept by the TTL job)
```

5. Why this beats `DELETE FROM ... WHERE created < ...`: the TTL job is bounded, monitored as a job, and runs without your application code remembering to schedule it.

Optional — If Time Allows

These parts are not required to complete the lab; they extend it by about 25 minutes. Do them if you finish early, or after class — the cluster and data from the core parts carry over.

Part F: A Code-Review Checklist for PKs (10 min)

Build a mental shortcut. For each of the following CREATE TABLE statements, predict (out loud or in a comment) whether it will hotspot, and if so, what to fix. Then verify your prediction by creating the table and inserting 5,000 rows.

```
-- 1
CREATE TABLE tbl1 (
  id SERIAL PRIMARY KEY,
  body STRING
);

-- 2
CREATE TABLE tbl2 (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  body STRING
);

-- 3
CREATE TABLE tbl3 (
  tenant UUID,
  ts TIMESTAMPTZ,
  body STRING,
  PRIMARY KEY (tenant, ts)
);

-- 4
CREATE TABLE tbl4 (
  region STRING,           -- only ~5 possible values
  ts TIMESTAMPTZ,
  body STRING,
  PRIMARY KEY (region, ts)
);

-- 5 (does this even create?)
CREATE TABLE tbl5 (
  ts TIMESTAMPTZ,
  shard INT AS (mod(extract(epoch from ts)::INT, 16)) STORED,
  body STRING,
  PRIMARY KEY (shard, ts)
);

-- 5b the same idea, expressed the way CockroachDB actually supports
CREATE TABLE tbl5b (
  ts TIMESTAMPTZ,
  body STRING,
  id UUID DEFAULT gen_random_uuid(),
  PRIMARY KEY (ts, id) USING HASH WITH (bucket_count = 16)
);
```

Expected answers: 1 hotspots (sequential). 2 distributes well. 3 hotspots per-tenant for a hot tenant (use hash sharding). 4 hotspots heavily (low-cardinality lead column) — bad.

5 is a trick question: it does not create at all.

```
ERROR: mod(): extract(): context-dependent operators are not allowed in STORED COMPUTED COLUMN
```

`extract(epoch from ts)` on a `TIMESTAMPTZ` depends on the session time zone, and a stored computed column must be deterministic forever — the value is written to disk once. Any `TIMESTAMPTZ → INT/STRING` conversion has the same problem (`to_char(ts AT TIME ZONE 'UTC')` is the escape hatch when you truly need one).

5b is what you actually want: `USING HASH` builds and maintains the shard column for you, with no hand-written expression to get wrong. Hand-rolled shard columns were the pattern before `USING HASH` existed; there is no reason to write one now.

Part H: Outbox Pattern with CDC (Playbook #8) (15 min)

The classic dual-write problem: your app writes to the database AND publishes a message to Kafka. If the process dies between the two, the systems disagree. The outbox pattern fixes this by making the publish part of the same database transaction.

1. Create the business table and the outbox:

```
CREATE TABLE orders_v2 (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  customer    STRING NOT NULL,  
  total       DECIMAL(12,2) NOT NULL,  
  placed      TIMESTAMPTZ DEFAULT now()  
);  
  
CREATE TABLE events_outbox (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  topic       STRING NOT NULL,  
  payload     JSONB NOT NULL,  
  created     TIMESTAMPTZ DEFAULT now()  
) WITH (ttl_expire_after = '7 days', ttl_job_cron = '@hourly');
```

2. Atomic write pattern — business data + event in one transaction:

```

BEGIN;
WITH new_order AS (
  INSERT INTO orders_v2 (customer, total)
  VALUES ('Alice', 99.99)
  RETURNING id
)
INSERT INTO events_outbox (topic, payload)
SELECT 'orders.created',
       jsonb_build_object('order_id', id, 'customer', 'Alice', 'total', 99.99)
FROM new_order;
COMMIT;

```

(A CTE carries the new id into the second insert. The CockroachDB shell has no psql-style `\gset` variables — the single statement is also what keeps the pair atomic.) If the COMMIT succeeds, both rows are durable. If anything fails, NEITHER is visible. Atomicity by definition — no dual-write inconsistency.

3. Hand off via CDC — in production, a changefeed ships the outbox to Kafka:

```

-- (demo mode includes an enterprise license; otherwise use a Core changefeed)
CREATE CHANGEFEED FOR TABLE events_outbox
  INTO 'kafka://broker:9092'
  WITH updated, resolved = '10s';

```

For this lab, exercise the Core variant — it emits to the SQL session, no broker needed:

```

-- In a second SQL session:
EXPERIMENTAL CHANGEFEED FOR events_outbox;

```

Then, in your main session, insert another order with its outbox event and watch the JSON appear in the second session.

4. Why it works:

- The business row and the event are written in the same transaction → atomicity.
- CockroachDB's MVCC ensures the changefeed sees only committed rows → no dirty publishes.
- Downstream consumers see at-least-once delivery, so they must be idempotent (use the event UUID as a dedupe key).
- TTL on the outbox keeps it from growing forever — events expire after the consumer has had ample time to read them.

5. Watch out:

- Outbox is at-least-once, not exactly-once. Downstream code must dedupe.
- Don't use the outbox as your queryable event store — it's a publishing buffer, not a system of record.

Cleanup

```
DROP DATABASE hotspots CASCADE;
```

`\q` exits the SQL shell; the cluster keeps running.

The cluster keeps running between labs — that is the point of it being persistent. To wipe everything and start fresh at any time:

```
scripts/crdb reset
```

Lab 3 Deliverables

□ **Hotspot observed:** `SERIAL` PK creates a single rightmost-range hotspot □ **UUID fix:** random PK distributes across many ranges □ **Composite PK** (Playbook #3): per-tenant co-location verified via `SHOW RANGES` and `EXPLAIN` □ **Hash sharding** (Playbook #1): single-tenant hot-key dispersed across 16 buckets □ **Pre-splits + bulk import** (Playbook #10): seeded a sequential-key table with 10 ranges □ **PK review checklist:** predicted hotspot risk for 5 PK shapes and verified □ **Append-Only Event Log + TTL** (Playbook #2, #9): table sweeps expired rows via a TTL job □ **Outbox Pattern** (Playbook #8): atomic business-write + event publish; CDC handoff verified

Challenge Exercises

1. **Find the optimal `bucket_count`**. Take the hot-account workload from Part D and re-run with bucket counts 1, 4, 16, 64, 256. Measure insert throughput at each. Plot if you like. Where does the curve plateau on your laptop?
2. **`crdb_internal_region` as a hash key**. Read up on multi-region locality (lab 7 covers it). Without changing anything else, would adding a `region` column to the hash composite primary key in Part D give better or worse distribution? Why?
3. **Convert without downtime**. You inherit `events_serial` in production. Design a zero-downtime migration to `events_uuid` (or to a hash-sharded scheme). What schema steps, what app-side code, what rollback plan?

Reference

Pattern	When to use	When to avoid
<code>id SERIAL PRIMARY KEY</code>	Almost never — replaced by UUID	Time-clustered writes; high concurrency
<code>id UUID PRIMARY KEY DEFAULT gen_random_uuid()</code>	Default for write-heavy tables	When you need ordered iteration by insert order
<code>PRIMARY KEY (tenant, id)</code>	Multi-tenant SaaS	When one tenant dominates traffic
<code>PRIMARY KEY (tenant, ts) USING HASH</code>	Time-series per tenant; "recent events"	When you NEVER read in ts order
<code>WITH (ttl_expire_after = '...')</code>	Append-only logs; sessions; ephemeral data	Long-lived authoritative records
Outbox + CDC	Publishing events alongside business writes	Outbox as a queryable system of record
Sequence-backed integers	Business-required monotonic IDs	Anything else — slow at scale
Pre-splits	Bulk imports of known-sequential keys	Random keys (they self-split)

For the full catalog with anti-patterns and a decision tree, see [SCHEMA_PATTERNS_PLAYBOOK.md](#).